# SET

Sets are used to store multiple items in a single variable.

. Important Properties

- No duplicates: Duplicate values are automatically removed.

- Unordered: Items do not have a fixed position.

- Mutable: You can add or remove items.

- Unindexed: You cannot access items using an index.

- Different data types: A set can contain integers, strings, and Boolean values.

## 1.add()

- add one item

In [7]:

```
fruits = {"apple","mango","orange"}
fruits.add("kiwi")
print(fruits)
```

```
{'mango', 'kiwi', 'orange', 'apple'}
```

## 2.update()

-add multiple item in one set

In [11]:

```
fruits = {"apple","mango","banana"}
colour = {"red","green","yellow"}
fruits.update(colour)
print(fruits)
```

```
{'banana', 'mango', 'apple', 'green', 'yellow', 'red'}
```

## 3.remove()

- remove specifice item in a set only value not index

In [14]:

```
num = {10,20,25,30,40,50}
num.remove(25)
print(num)
```

```
{50, 20, 40, 10, 30}
```

In [17]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
num = {10,20,25,30,40,50}
num.remove(60)
print(num)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[17], line 2
      1 num = {10,20,25,30,40,50}
----> 2 num.remove(60)
      3 print(num)

KeyError: 60
```

# 4.discard()

-remove item but not show error while remove invalid value

In [24]:

```
num = {10,20,25,30,40,50}
num.discard(25)
print(num)
```

```
{50, 20, 40, 10, 30}
```

In [26]:

```
num = {10,20,25,30,40,50}
num.discard(60)
print(num)
```

```
{50, 20, 40, 25, 10, 30}
```

# 5.pop()

-it remove any value not specific value

In [31]:

```
num = {10,20,25,30,40,50}
num.pop()
print(num)
```

```
{20, 40, 25, 10, 30}
```

# 6.access set

using in,not in, for

In [34]:

```
num = {10,20,25,30,40,50}
print(10 in num)
```

```
True
```

In [36]:

```
num = {10,20,25,30,40,50}
print(1 not in num)
```

```
True
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# 7.clear()

-remove all value in set

In [39]:

```
num = {10,20,25,30,40,50}
num.clear()
print(num)
```

```
set()
```

# 8.del

-delete entier variable

In [42]:

```
num = {10,20,25,30,40,50}
del num
```

# 9.copy()

- copy one variable to another

In [45]:

```
a={1,2,3}
b=a.copy()
print(b)
```

```
{1, 2, 3}
```

# 10.Union

-The union() method returns a new set with all items from both sets.

In [52]:

```
a={1,2,3,4,5}
b={6,7,8,9,10}
c=a.union(b)
print(c)
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

In [54]:

```
a={1,2,3,4,5}
b={6,7,8,9,10}
c=a | b
print(c)
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

In [56]:

```
#join multiple set
name ={"Reeta"}
age={20}
degree={"BE"}
cgpa = {8.51}
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
data = name.union(age,degree,cgpa)
print(data)
```

```
{8.51, 20, 'BE', 'Reeta'}
```

In [58]:

```
name ={"Reeta"}
age={20}
degree={"BE"}
cgpa = {8.51}
data = name|age|degree|cgpa
print(data)
```

```
{8.51, 20, 'BE', 'Reeta'}
```

# 11.Intersection

Keep ONLY the duplicates

The intersection() method will return a new set, that only contains the items that are present in both sets.

In [61]:

```
a={34,56,12,78,99}
b={99,34,11,45,67}
c=a.intersection(b)
print(c)
```

```
{34, 99}
```

In [63]:

```
a={34,56,12,78,99}
b={99,34,11,45,67}
c=a & b
print(c)
```

```
{34, 99}
```

In [65]:

```
a={34,56,12,78,99}
b={99,34,11,45,67}
a.intersection_update(b)
print(a)
```

```
{34, 99}
```

# 12.Difference

The difference() method will return a new set that will contain only the items from the first set that are not present in the other set.

In [68]:

```
a={34,56,12,78,99}
b={99,34,11,45,67}
c=a.difference(b)
print(c)
```

```
{56, 12, 78}
```

In [70]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
a={34,56,12,78,99}
b={99,34,11,45,67}
c=a - b
print(c)
```

```
{56, 12, 78}
```

In [74]:

```
a={34,56,12,78,99}
b={99,34,11,45,67}
a.difference_update(b)
print(a)
```

```
{56, 12, 78}
```

# 13.Symmetric Differences

The symmetric_difference() method will keep only the elements that are NOT present in both sets.

In [79]:

```
a={1,2,3,4,5}
b={3,1,7,8,6}
c=a.symmetric_difference(b)
print(c)
```

```
{2, 4, 5, 6, 7, 8}
```

In [81]:

```
a={1,2,3,4,5}
b={3,1,7,8,6}
c=a ^ b
print(c)
```

```
{2, 4, 5, 6, 7, 8}
```

In [83]:

```
a={1,2,3,4,5}
b={3,1,7,8,6}
a.symmetric_difference_update(b)
print(a)
```

```
{2, 4, 5, 6, 7, 8}
```

# 14.frozenset

## - we cannot add and remove in frozenset

frozenset is used when you need a set whose items cannot be changed. You cannot add or remove items after creating it.

In [87]:

```
a=frozenset({1,2,3,4})
print(a)
print(type(a))
```

```
frozenset({1, 2, 3, 4})
<class 'frozenset'>
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Practice Question

## 1. Write a Python program to remove duplicate elements from a list using a set.

In [92]:

```
a={1,1,2,3,4,2,4}
print(a)
```

```
{1, 2, 3, 4}
```

## 2. Write a Python program to find common elements between two sets.

In [95]:

```
a={1,4,6,2,8,6}
b={1,6,4,3,2,9}
c=a.intersection(b)
print(c)
```

```
{1, 2, 4, 6}
```

## 3. Write a Python program to find the union of two sets.

In [98]:

```
a={1,2,3,4,5,6,7}
b={"a","b","c"}
c=a.union(b)
print(c)
```

```
{1, 2, 3, 4, 5, 6, 7, 'c', 'b', 'a'}
```

## 4. Write a Python program to find elements present in the first set but not in the second set.

In [105]:

```
a={1,4,6,2,8,6}
b={1,3,2,9}
c=a.difference(b)
print(c)
```

```
{8, 4, 6}
```

## 5. Write a Python program to add and remove elements from a set.

In [114]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
a={1,2,3,4}
a.add(5)
print(a)
```

```
{1, 2, 3, 4, 5}
```

In [116]:

```
a={1,2,3,4}
a.remove(1)
print(a)
```

```
{2, 3, 4}
```

## 6. Write a Python program to check whether one set is a subset of another set.

In [127]:

```
a = {1, 2}
b = {1, 2, 3, 4}
print(a.issubset(b))
```

```
True
```

## 7. Write a Python program to convert a list into a frozenset and display the result.

In [130]:

```
l=[1,2,3,4]
f=frozenset(l)
print(f)
```

```
frozenset({1, 2, 3, 4})
```

## 8. Write a Python program to create a frozenset and demonstrate why its elements cannot be added or removed.

In [135]:

```
a=frozenset({1,2,3,4})
print(a)
```

```
frozenset({1, 2, 3, 4})
```

## 9. Write a Python program to find the symmetric difference between two frozenset objects.

In [154]:

```
a=frozenset({1,2,3,4})
b=frozenset({3,4})
c=a.symmetric_difference(b)
print(c)
```

```
frozenset({1, 2})
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# 10.Write a Python program to check whether two sets are disjoint using the isdisjoint() method.

In [157]:

```
a = frozenset({1, 2})
b = frozenset({3, 4})
c = frozenset({2, 3})
print(a.isdisjoint(b))
print(a.isdisjoint(c))
```

```
True
False
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF